

claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-a42d8a73cab888a07.jsonl`
- SHA-256: `267e29c8d70e8c4ed2d0fea0075bbbe8c5df9d5bf012cb7168ee3569f5d0b354`
- Source modified: `2026-07-02T03:00:39+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_41c8dc8a-8a4`
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`
```

Transcript

1. user (2026-07-02T02:59:48.415Z)

You are an adversarial verifier in a tech-debt audit. Your default stance: REFUTE. Today is 2026-07-02.

A finder claims the following about the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer:

```
{
  "title": "GDriveApiStorage.fetch_to_local buffers the entire file in RAM
(get_media().execute()); put() deletes the old object before the new upload succeeds",
  "category": "critical-fix",
  "severity": "P2",
  "file": "backend/storage/gdrive_api.py",
  "line": 144,
  "evidence": "gdrive_api.py:143-148: `data =
self._service.files().get_media(fileId=file_id).execute()` then `fh.write(data)` ? the whole
object is materialized as one bytes value (no MediaIoBaseDownload streaming).
gdrive_api.py:163-170: put() resolves an existing file at the path and `files().delete(...)`
BEFORE `files().create(...)` ? delete-then-create is not atomic.",
  "why_it_matters": "Latent OOM + data-loss on the Drive-API storage hot path before it goes
live for the reel round-trip.",
  "proposed_fix": "Stream downloads with googleapiclient.http.MediaIoBaseDownload in chunks; for
put(), create the new file first (or upload to a temp name) and delete the old one only after
success.",
  "effort": "S",
  "tracked_in": ""
}
```

Repo root candidates: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer, C:/Users/avidu/Projects/khelsutra-guru/khelsutra, C:/Users/avidu/Projects/khelsutra-guru/sports-data-collector, C:/Users/avidu/Projects/khelsutra-guru/rally-annotator, C:/Users/avidu/Projects/khelsutra-guru/sports-obsreport (pick from the finding's repo field; non-git dirs: 'C:/Users/avidu/Projects/khelsutra-guru/Annotation Setup', C:/Users/avidu/Projects/khelsutra-guru/khelsutra-screenshot).

VERIFY against the CURRENT working tree (just pulled to origin master/main): (1) does the cited code actually exist and behave as claimed at file:line? (2) was it already fixed by a recent commit (git log --oneline -15 -- <file>)? (3) is the severity honest - construct the concrete failure scenario or downgrade; (4) is the proposed fix sane and correctly scoped; (5) is it owner-gated per: alpha/serving go-live, Studio P10/P11 corpus-promotion/training, product/strategy/licensing, real cloud spend, counsel ToS/DPA?

HARD RULES: READ-ONLY (no file modifications, no installs, no test-suite runs). gh read-only commands allowed. Your reasoning MUST cite file:line you actually read. CONFIRMED requires you personally saw the defect/debt in current code; if you cannot reproduce the claim from the code, say REFUTED; if a commit fixed it, ALREADY_FIXED.

2. user (2026-07-02T02:59:53.592Z)

```

1      """Google Drive **API** backend (OAuth / service-account) ? ``gdrive-api:`` (M12, D14
Northstar).
2
3      The cloud-native Drive round-trip: read a source from + write the stitched reel **back
to** a user's
4      Google Drive **via the Drive API** ? no local mount required (unlike the ``gdrive:``
backend, which
5      is plain local-FS over a *mounted* Drive for Desktop and stays untouched). This is the
headless path a
6      Cloud Run / GPU box uses.
7
8      ``gdrive-api://<path>`` names a path **relative to a root folder** (default the Drive
``root``, or
9      ``storage.gdrive-api.root_folder_id``); the backend resolves each path component to a
Drive file id via
10     ``files().list``. The Google libraries (``google-api-python-client`` + ``google-auth``)
are imported
11     **lazily** ? ``import backend.storage`` never pulls them ? and a ``service`` may be
**injected** for
12     fully-offline tests (mirroring ``GCSStorage(client=...)``).
13
14     ? **Credentials are never in config.** The real service is built from Application
Default Credentials
15     (a service account / workload identity) over the ``drive`` scope. **Per-user interactive
OAuth (the
16     consumer "connect your Drive" flow: client id/secret + refresh tokens) is OWNER-gated**
? this backend
17     ships the service-account/ADC build + the injected fake; the live per-user OAuth app is
a follow-up.
18
19     **Scope caveats (M12):** handles **binary** files (videos / reels / JSON) ? Google-
native Docs
20     (Sheets/Slides) have no direct download and would need ``files().export_media`` (out of
scope; export
21     them first). ``put`` is idempotent (deletes an existing file at the same path before
creating). Path
22     resolution keys on ``(name, parent)``; if Drive already holds duplicate-named files
under one folder
23     (a rare pre-existing state this backend never creates), resolution returns the first
match.
24     """
25
26     from __future__ import annotations
27
28     import io
29     import os
30     from datetime import datetime
31     from typing import Any, Iterator, Optional
32
33     from backend.storage.base import StorageBackend
34     from backend.storage.ref import StorageRef, StorageStat
35
36     _FOLDER_MIME = "application/vnd.google-apps.folder"
37     _DRIVE_SCOPE = "https://www.googleapis.com/auth/drive"
38
39
40     def _q_escape(name: str) -> str:
41         """Escape a value for a Drive ``q`` string literal (single-quoted): ``\`` then
42         """
43         return name.replace("\\", "\\").replace("'", "\'")
44
45     class GDriveApiStorage(StorageBackend):
46         """Drive-API-backed storage. Construct with an injected ``service`` (tests) or let
it build one

```

```

47         from ADC (production). ``root_folder_id`` anchors relative ``gdrive-api://``
paths. """
48
49     def __init__(
50         self, *, service: Any = None, root_folder_id: str = "root", **_ignored
51     ) -> None:
52         self._root_id = root_folder_id or "root"
53         if service is not None:
54             self._service = service
55             return
56         self._service = self._build_service()
57
58         # -- construction (real path; lazy Google imports)
59         -----
60         def _build_service(self) -> Any:
61             try:
62                 import google.auth # type: ignore
63                 from googleapiclient.discovery import build # type: ignore
64             except (
65                 ImportError
66             ) as e: # pragma: no cover - exercised only without the extra installed
67                 raise RuntimeError(
68                     "the gdrive-api backend needs the 'storage-gdrive-api' extra "
69                     "(google-api-python-client + google-auth). Install it on a host that
70                     uses "
71                     "gdrive-api:// URIs; credentials come from ADC (a service account /
72                     workload "
73                     "identity over the Drive scope) ? never from config."
74                 ) from e
75             creds, _ = google.auth.default(scopes=[_DRIVE_SCOPE])
76             return build("drive", "v3", credentials=creds, cache_discovery=False)
77
78         def _media_body(self, local_path: str, content_type: Optional[str]) -> Any:
79             """The upload media object for ``files().create`` ? lazy ``MediaFileUpload``
80             (real path).
81             A test seam: monkeypatch this to avoid the ``googleapiclient`` dependency."""
82             from googleapiclient.http import (
83                 MediaFileUpload, # type: ignore # pragma: no cover
84             )
85
86             return MediaFileUpload(
87                 local_path, mimetype=content_type, resumable=True
88             ) # pragma: no cover
89
90         # -- path -> id resolution
91         -----
92         def _resolve_id(self, key: str, *, create_dirs: bool = False) -> Optional[str]:
93             """Walk ``key``'s path components from the root folder, returning the leaf's
94             file id (or
95             ``None`` if a component is missing). With ``create_dirs`` EVERY missing
96             component is created as
97             a folder ? ``put`` uses this on the parent-directory path (where all components
98             are folders),
99             so a write into a fresh subfolder works. The read paths leave it ``False`` (a
100             missing component
101             ? ``None``), never creating anything."""
102             parent = self._root_id
103             parts = [p for p in key.split("/") if p]
104             for part in parts:
105                 q = (
106                     f"name = '{_q_escape(part)}' and '{_q_escape(parent)}' in parents "
107                     f"and trashed = false"
108                 )
109                 found = (
110                     self._service.files()

```

```

102         .list(q=q, spaces="drive", fields="files(id, mimeType)")
103         .execute()
104         .get("files", [])
105     )
106     if found:
107         parent = found[0]["id"]
108     elif create_dirs:
109         parent = (
110             self._service.files()
111             .create(
112                 body={
113                     "name": part,
114                     "mimeType": _FOLDER_MIME,
115                     "parents": [parent],
116                 },
117                 fields="id",
118             )
119             .execute()["id"]
120         )
121     else:
122         return None
123     return parent
124
125     def _split(self, key: str) -> tuple[str, str]:
126         """Split a `gdrive-api://` key into (parent dir path, leaf name)."""
127         clean = key.strip("/")
128         head, _, name = clean.rpartition("/")
129         return head, name
130
131     # -- StorageBackend interface
132     -----
133     def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
134         if dst is None:
135             raise ValueError(
136                 "GDriveApiStorage.fetch_to_local requires a dst path (no local mount)"
137             )
138         dst = str(dst)
139         if os.path.exists(dst) and not overwrite:
140             return dst
141         file_id = self._resolve_id(ref.key)
142         if not file_id:
143             raise FileNotFoundError(f"gdrive-api: {ref.uri!r} not found")
144         data = (
145             self._service.files().get_media(fileId=file_id).execute()
146         ) # alt=media ? raw bytes
147         os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
148         with io.open(dst, "wb") as fh:
149             fh.write(data)
150         return dst
151
152     def put(self, local_path, ref, *, content_type=None) -> StorageRef:
153         parent_path, name = self._split(ref.key)
154         parent_id = (
155             self._resolve_id(parent_path, create_dirs=True)
156             if parent_path
157             else self._root_id
158         )
159         if parent_id is None:
160             raise FileNotFoundError(f"gdrive-api: cannot resolve parent of {ref.uri!r}")
161         # Idempotent: Drive allows duplicate (name, parent), so a plain create() on a
162         # ORPHAN the prior file (and reads would keep returning the stale first match).
163         # existing file at this exact path first, so put() overwrites rather than
164         # accumulates.

```

```

163         existing = self._resolve_id(ref.key)
164         if existing:
165             self._service.files().delete(fileId=existing).execute()
166         self._service.files().create(
167             body={"name": name, "parents": [parent_id]},
168             media_body=self._media_body(str(local_path), content_type),
169             fields="id",
170         ).execute()
171         return ref
172
173     def exists(self, ref) -> bool:
174         return self._resolve_id(ref.key) is not None
175
176     def stat(self, ref) -> StorageStat:
177         file_id = self._resolve_id(ref.key)
178         if not file_id:
179             raise FileNotFoundError(f"gddrive-api: {ref.uri!r} not found")
180         meta = (
181             self._service.files()
182             .get(fileId=file_id, fields="size, modifiedTime, md5Checksum, mimeType")
183             .execute()
184         )
185         modified = None
186         if meta.get("modifiedTime"):
187             # Drive returns RFC-3339 (e.g. "2026-06-22T10:00:00.000Z"); normalize the
trailing Z.
188             modified = datetime.fromisoformat(
189                 meta["modifiedTime"].replace("Z", "+00:00")
190             )
191         return StorageStat(
192             size=int(meta.get("size") or 0),
193             modified=modified,
194             etag=meta.get("md5Checksum"),
195             content_type=meta.get("mimeType"),
196         )
197
198     def list(self, prefix) -> Iterator[StorageRef]:
199         folder_id = (
200             self._resolve_id(prefix.key) if prefix.key.strip("/") else self._root_id
201         )
202         if not folder_id:
203             return
204         base = prefix.key.strip("/")
205         page_token: Optional[str] = None
206         while True:
207             resp = (
208                 self._service.files()
209                 .list(
210                     q=f"'{_q_escape(folder_id)}' in parents and trashed = false",
211                     spaces="drive",
212                     fields="nextPageToken, files(id, name)",
213                     pageToken=page_token,
214                 )
215                 .execute()
216             )
217             for f in resp.get("files", []):
218                 key = f"{base}/{f['name']}" if base else f["name"]
219                 yield StorageRef("gddrive-api", "", key, f"gddrive-api://{key}")
220             page_token = resp.get("nextPageToken")
221             if not page_token:
222                 break
223
224     def delete(self, ref) -> None:
225         file_id = self._resolve_id(ref.key)
226         if file_id:

```

```
227         self._service.files().delete(fileId=file_id).execute()
228
```

3. user (2026-07-02T02:59:55.735Z)

```
b6214e5 chore: ruff format the whole repo (259 files) (#387)
f523d3a refactor: enable isort (I) in ruff, auto-fix 143 files (Q6) (#360)
819e395 feat(serving): M12 ? gdrive-api:// OAuth/API StorageBackend (D14 Northstar) (#338)
master
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
```

4. user (2026-07-02T03:00:39.292Z)

Structured output provided successfully